

Неделя 1: Что такое архитектура ПО?

Архитектура ПО / Software Architecture

Цели сегодняшней лекции

К концу лекции вы сможете:

1. Объяснить простыми словами **архитектуру** (*architecture*), **проектирование** (*design*) и **код** (*code*)
2. Сказать, что курс *будет* и *не будет* покрывать
3. Назвать, кто такие **заинтересованные стороны** (*stakeholders*) и почему им важна структура
4. Указать **части и связи** в привычных системах (чат, портал, магазин)

Представьте школьное здание:

- **Архитектура** (*architecture*): где корпуса, сколько этажей, где лестницы и выходы — потом менять дорого
- **Проектирование (детали)** (*design*): расстановка парт, цвет дверей, точный план кабинета
- **Строительство / код** (*code*): заливка бетона, проводка, покраска — конкретная работа «руками»

В ПО похоже: одни решения задают всю систему; другие — локальная отделка.

Три уровня (рабочие определения)

Архитектура

Крупная структура: главные части, как они общаются, что трудно откатить.

Проектирование

Решения среднего уровня внутри части: классы, экраны, локальные паттерны.

Код

Конкретная реализация: строки, которые выполняет машина.

Границы на практике размыты — уровни это *инструмент мышления*, не закон.

«Структура, которую потом дорого менять»

Архитектура проявляется, когда изменение стоит дорого:

- Разделить одно большое приложение на два сервиса (или слить обратно)
- Перенести все пользовательские данные в другой тип хранилища
- Изменить, как мобильное приложение общается с сервером

Частая ошибка

Называть архитектурой каждое имя класса или цвет кнопки. Если легко переделать за полдня — чаще это проектирование или код.



Хороший архитектор тоже пишет (или читает) код — но часто «отъезжает камеру».

Одна функция — три взгляда

Пример: «пользователи могут отправить фото в чат»

- **Архитектура:** медиа на сервере чата или в отдельном файловом сервисе? Кто может скачать?
- **Проектирование:** как устроен поток экрана загрузки? Кнопка повтора? Прогресс?
- **Код:** точные HTTP-вызовы, проверка файла, тексты ошибок

Викторина

Какое решение больше всего похоже на **архитектуру**?

1. Переименовать переменную из `x` в `userCount`
2. Выбрать цвет кнопки «Отправить»
3. Решить, что мобильное приложение будет общаться с центральным сервером, который владеет всеми сообщениями
4. Исправить опечатку в комментарии

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

3. Владение сообщениями на стороне клиент–сервер задаёт всю систему и дорого откатывается.

(1) и (4) — уровень кода; (2) — в основном UI-проектирование.

Что курс *будет* покрывать

- Атрибуты качества (*quality attributes*) (производительность, надёжность, безопасность, ...) простым языком
- Стили: монолит vs модульность, клиент–сервер, слои
- MVC / MVP на **высоком уровне**; API как контракт; хранилища данных «с высоты птичьего полёта»
- Простые диаграммы (C4-lite / эскизы компонентов)
- Основы развёртывания; компромиссы; кейсы; этика и надёжность
- Мини-проект: задача → качества → эскиз → обоснование

Во что мы *не* углубляемся

- Формальные языки описания архитектуры и «тяжёлый» UML-сертификационный трек
- Корпоративные фреймворки (церемонии в духе TOGAF)
- Глубокие доказательства по распределённым системам и оркестрация микросервисов
- Мода «всегда берите самый новый модный термин»

Частая ошибка

Думать, что архитектура = выучить каталог паттернов наизусть. Нам нужны понятные схемы и честные компромиссы.

Как выглядит успех через несколько недель

Вы сможете:

1. Отличать разговор об архитектуре от «как работает одна функция»
2. Назвать заинтересованные стороны и несколько целей качества для небольшого продукта
3. Нарисовать схему частей и связей, понятную одnogруппнику

Узкие спецкурсы — потом; сначала фундамент.

Вопрос 2 — Рамки курса (верно/неверно)

Викторина

Верно или неверно?

Этот курс сосредоточен на доказательствах теорем о распределённых системах и формальных языках архитектуры.

1. Верно
2. Неверно

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

Неверно. Мы остаёмся практичными: интуиция, диаграммы, компромиссы и кейсы — не тяжёлые формальные методы.

Кому важна архитектура?

Заинтересованная сторона (*stakeholder*) — любой, на кого влияет структура или поведение системы.

- **Пользователи** — работает ли, когда нужно?
- **Разработчики** — можно ли безопасно менять?
- **Операторы** — можно ли запускать и мониторить?
- **Бизнес / школа** — стоимость, риск, сроки

Разным людям важны разные качества. Архитектура их балансирует.

Одна система — разные заботы

Пример школьного портала:

- Студент: «Могу ли сдать домашку с телефона?»
- Учитель: «Оценки скрыты от других студентов?»
- IT: «Сможем восстановить бэкап после сбоя?»
- Директор: «Будет ли работать через год, когда вырастем?»

Запомните

Хороший разговор об архитектуре начинается с вопроса: **чью проблему мы решаем?**

Мы потренируем мышление о стейкхолдерах

В следующих неделях: сценарии качества обычными предложениями.

Сегодня: заметить, что «хорошая архитектура» — не вкус одного человека.

Вопрос 3 — Заинтересованные стороны

Викторина

Кто *менее всего* похож на прямую заинтересованную сторону приложения заказа еды в столовой?

1. Студент, который заказывает обед
2. Сотрудники кухни, готовящие заказы
3. Разработчик, поддерживающий приложение
4. Турист, который никогда не бывает в кампусе и не пользуется приложением

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

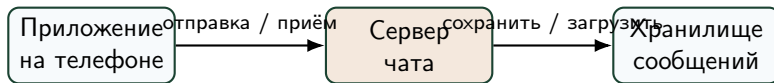
4. У кого нет ни использования, ни ответственности за систему, тот здесь не стейкхолдер. Пользователи, персонал, разработчики и операторы — обычно да.

Для любого приложения спросите:

1. Какие **главные части**? (приложение на телефоне, сайт, сервер, БД, ...)
2. Как части **общаются**? (запросы, сообщения, синхронизация)
3. Где живут **данные**?
4. Что будет, если одна часть **недоступна**?

Это уже архитектурное мышление — до любых модных названий паттернов.

Пример А — Мессенджер



- Части: клиенты, сервер, хранилище
- Связь: сетевые запросы (и, возможно, push-уведомления)
- Дорогое изменение: перенос сообщений в новое хранилище

Типичные части (упрощённо):

- Браузер / мобильный клиент
- Веб-приложение (страницы + вход)
- База данных пользователей, курсов, оценок
- Возможно, файловое хранилище для загрузок

Архитектурные вопросы: кто видит оценки? Что, если БД тормозит в неделю экзаменов?

- Каталог, корзина, оплата, статусы доставки
- Часто: веб/приложение + бэкенд + БД товаров + платёжный провайдер

Попробуйте

На практике: выберите *одно* знакомое приложение и перечислите части и связи своими словами.

Викторина

На схеме мессенджера стрелка между приложением и сервером чата в основном означает:

1. Выбор цвета в CSS
2. Путь связи / обмена данными между частями
3. Цикл `for` в Python
4. Меню кампусной столовой

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Стрелки показывают, как части общаются — это сердце архитектурных эскизов.

Викторина

«Сделаем кнопку Отправить синей» лучше всего отнести к:

1. Архитектуре
2. В основном UI-проектированию (не труднооткатываемый структурный выбор)
3. Теории баз данных
4. Роли заинтересованной стороны

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Цвет кнопки — локальный UI-дизайн; его легче менять, чем топологию серверов.

1. Архитектура = крупная структура и труднооткатываемые решения
2. Проектирование и код ближе к деталям
3. Курс практический: схемы, качества, компромиссы — не формальные доказательства
4. Заинтересованные стороны (пользователи, разработчики, операторы, бизнес) заботятся о разном
5. Привычные приложения — нормальный материал для архитектурных вопросов

архитектура (*architecture*) · проектирование (*design*) · код (*code*)
заинтересованные стороны (*stakeholders*) · части · связи

Добавьте каждое в глоссарий с кратким пояснением и примером.

Что ещё мутно: архитектура vs проектирование, заинтересованные стороны или примеры?